

Pomelo ESP32-C3 Firmware Architecture Plan

Based on bGeigieZen Worker/Handler/Controller Pattern

Executive Summary

Port Pomelo scintillation detector firmware from ATSAML21G18B (ARM Cortex-M0+) to ESP32-C3 (RISC-V) using the proven bGeigieZen architecture pattern. Maintain **100% backwards compatibility** with existing JSON command protocol and NVM parameters while adding WiFi upload, Bluetooth reporting, and SD card logging.

Target: ESP32-C3 DevKit + Pomelo sensor hardware
Timeline: 10 weeks (7 implementation phases)
Framework: ESP-IDF with Arduino component (PlatformIO)

Current State Analysis

Pomelo Firmware (ATSAML21G18B)

- **Architecture:** Monolithic main.c (2,219 lines), ISR-driven
- **Critical ISR:** `trigger_callback()` on PA18 - gamma detection with <1µs ADC readout
- **State Machine:** Implicit (daq_enabled, sys_power flags)
- **Parameters:** Two NVM structures (core_params, physics_params)
- **Interface:** JSON commands over USB/UART (921600 baud)
- **Key Modules:** HV control with temp compensation, spectral deconvolution (1024 ch), coincidence detection

bGeigieZen Architecture (ESP32 M5Stack)

- **Pattern:** Worker/Handler/Controller with central orchestration
 - **Workers (13):** Modular data acquisition (buttons, GM sensor, GPS, battery, RTC, storage, config)
 - **Handlers (6):** Data processing outputs (SD loggers, Bluetooth, API connector)
 - **Controller:** Central state machine, worker/handler registration and activation
 - **State Management:** DeviceState struct with modes, connectivity status
 - **Flow:** Polling-based `produce_data()` returning worker status (idle/data_read)
-

Architecture Design

1. ESP32-C3 Platform Mapping

ATSAML21	Pomelo Use	ESP32-C3 Equivalent	Notes
EXTINT2 (PA18)	Gamma trigger	GPIO interrupt	<code>gpio_isr_handler_add()</code> , IRAM_ATTR required
ADC (12-bit)	Energy	ADC1 (12-bit)	<code>adc1_get_raw()</code> in ISR
DAC (2ch, 10-bit)	HV + threshold	DAC (2ch, 8-bit)	Reduced resolution
TCC0 PWM	HV boost	LEDc peripheral	<code>ledc_set_duty()</code>
TC0 Timer	Time base	GPTimer	<code>gptimer_new_timer()</code>
I2C	Temp sensors	I2C master	PCT2075/TMP116/TMP451
USB CDC	Commands	TinyUSB	Native USB-CDC
NVM	Parameters	NVS	<code>nvs_get_blob()</code> / <code>nvs_set_blob()</code>
CCL Logic	Coincidence	Software	No hardware CCL on ESP32-C3

Critical Constraint: ISR timing <10μs (ESP32-C3 vs <5μs ATSAML21)

2. Worker Architecture (10 Workers)

Worker ID	Class	Data Type	Purpose
k_worker_gamma_detector	GammaDetectorWorker	GammaData	ISR wrapper, spectral processing, FIFO
k_worker_hv_controller	HvControllerWorker	HvData	HV regulation, temp compensation
k_worker_temp_sensor	TempSensorWorker	TempData	Auto-detect PCT2075/TMP116/TMP451
k_worker_usb_serial	UsbSerialWorker	SerialCommand	JSON command interface
k_worker_wifi_connector	WiFiConnectorWorker	WiFiStatus	WiFi connection management
k_worker_bt_connector	BluetoothWorker	BtStatus	BLE advertising
k_worker_sd_interface	SdInterfaceWorker	SdStatus	SD card mount/status
k_worker_coincidence	CoincidenceWorker	CoincidenceData	Software coincidence logic
k_worker_device_state	PomeloController	DeviceState	Central state machine
k_worker_local_storage	PomeloStorage	bool	NVS parameter persistence

Base Pattern: All workers inherit from `ProcessWorker<T>` (SensorReporter library)

3. Handler Architecture (5 Handlers)

Handler ID	Class	Purpose	Data Source
k_handler_sd_logger	SdLoggerHandler	Spectrum + list mode to SD	GammaDetectorWorker
k_handler_wifi_uploader	WiFiUploaderHandler	Upload to Safecast API	GammaDetectorWorker
k_handler_bt_reporter	BluetoothHandler	BLE data streaming	GammaDetectorWorker
k_handler_usb_serial_cmd	UsbSerialCmdHandler	JSON command responses	UsbSerialWorker
k_handler_display	DisplayHandler	OLED/LCD display (optional)	All workers

Base Pattern: All handlers inherit from `Handler` (SensorReporter library)

4. Controller: PomeloController

DeviceState Structure:

```
struct DeviceState {
    enum PowerState { e_power_off, e_power_standby, e_power_hv_armed,
e_power_running };
    enum Mode { e_mode_spectrum, e_mode_list, e_mode_dosimetry };

    bool initialized;
    PowerState power_state;
    Mode mode;
    bool hv_enabled, daq_enabled, coincidence_enabled, temp_valid;
    uint8_t output_flags; // Routing bits from NVM
    bool sd_card_ready, wifi_connected, bt_connected, usb_connected;
};
```

Key Methods:

- `initialize()` - System setup
- `start_default_workers()` - Activate core workers
- `power_on()` / `power_off()` - HV control
- `daq_start()` / `daq_stop()` - Data acquisition control
- `route_outputs()` - Enable handlers based on output_flags

5. Critical Path: ISR Integration

Challenge: `trigger_callback()` ISR is timing-critical (<1µs ADC readout, 5-cycle peak detector reset)

Solution: Two-Stage Processing

Stage 1: ISR (IRAM, ultra-fast)

```
static void IRAM_ATTR gamma_trigger_isr(void* arg) {
    // 1. ADC read (~1µs)
    uint16_t adc_val = adc1_get_raw(ADC1_CHANNEL_0);

    // 2. Peak detector reset (critical timing)
    gpio_set_level(PIN_PEAK_RESET_1, 1);
    gpio_set_level(PIN_PEAK_RESET_2, 1);
    asm volatile("nop; nop; nop; nop; nop;");
    gpio_set_level(PIN_PEAK_RESET_1, 0);
    gpio_set_level(PIN_PEAK_RESET_2, 0);

    // 3. Push to lock-free ring buffer
    worker->ring_buffer.push({adc_val, esp_timer_get_time()});
}
```

Total ISR: ~3-5µs ✓

Stage 2: Worker (normal priority, deferred)

```
int8_t GammaDetectorWorker::produce_data(const worker_map_t& workers) {
    while (ring_buffer.available()) {
        auto event = ring_buffer.pop();

        // Spectral deconvolution (pMove matrix)
        uint16_t bin = apply_deconvolution(event.adc_value);

        // Update spectrum, list mode FIFO
        data.spectrum[bin]++;
        data.pulse_count++;
        push_to_list_fifo(event.adc_value, event.timestamp);
    }
    return e_worker_data_read;
}
```

Communication: Lock-free ring buffer (256 events, ISR-safe)

6. Backwards Compatibility

Command Interface: `CommandInterpreter` maps JSON commands to worker actions

- `h` → Get histogram (spectrum)
- `s` → System status
- `c` → Config
- `x` → Power on

- `z` → Power off
- `m` → Dosimetry (CPM + $\mu\text{Sv/h}$)
- `p<params>` → Set parameter

Parameter Structures: Preserve in NVS as binary blobs

```
struct core_params {
    uint8_t version;
    float vDac[2];           // HV DAC calibration
    float iMeas[3];          // Current measurement cal
    uint16_t threshold;      // Discriminator level
    uint8_t sys_outputs;     // Output routing flags
    uint8_t sys_pulseChar;   // Pulse character
    bool sys_power, sys_coincidence;
};

struct physics_params {
    uint8_t version;
    float sipm_vMin, sipm_vMax, sipm_v0deg, sipm_vTempComp;
    float ecal[3];           // Energy calibration polynomial
    float uSvph_constant;    // Dose conversion
    char detString[64];      // Detector ID
    uint8_t tempType;        // 1=TMP116, 2=TMP451, 3=PCT2075
};
```

Output Routing: `sys_outputs` byte controls data destinations

- Bit 0-3: UART (pulse, fast pulse, energy, energy+TS)
- Bit 4-6: USB (pulse, energy, energy+TS)
- Handlers activate based on flags

Implementation Phases (10 Weeks)

Phase 1: Scaffold + LED Blink (Week 1)

Goal: Verify build system, basic controller

Tasks:

1. Create PlatformIO project (ESP-IDF + Arduino)
2. Minimal PomeloController + DeviceState
3. LED blink on GPIO2
4. USB Serial "hello world"

Success: Compiles, uploads, blinks, prints to serial

Phase 2: Gamma Detector Worker + ISR (Week 2-3)

Goal: Core ISR functionality, spectral histogram

Tasks:

1. Implement GammaDetectorWorker with ISR
2. GPIO interrupt on PIN_GAMMA_TRIGGER
3. ADC configuration (12-bit, max speed)
4. Lock-free ring buffer (ISR→worker)
5. Port spectral deconvolution (pMove matrix)
6. Test with pulse generator

Success: ISR captures pulses, spectrum builds, <10μs ISR duration

Test Equipment: Function generator, oscilloscope

Phase 3: HV Control + Temp Compensation (Week 4)

Goal: High voltage regulation, temperature monitoring

Tasks:

1. HvControllerWorker with LEDC PWM
2. DAC for HV setpoint
3. TempSensorWorker (I2C auto-detect)
4. Temperature compensation algorithm
5. HV regulation loop tuning

Success: HV reaches setpoint (30-70V), ±0.1V stability, temp comp working

Test Equipment: Oscilloscope, multimeter, thermocouple

Phase 4: USB/Serial Command Interface (Week 5)

Goal: Full backwards compatibility with JSON commands

Tasks:

1. UsbSerialWorker (USB-CDC)
2. CommandInterpreter class
3. Port all JSON commands (h, s, c, x, z, m, p)
4. PomeloStorage (NVS) for parameters
5. Test parameter save/load

Success: All JSON commands work, parameters persist across reboots

Test: Existing Pomelo Python/GUI tools

Phase 5: SD Card Logging (Week 6)

Goal: Spectral and list mode logging

Tasks:

1. SdInterfaceWorker
2. SdLoggerHandler
3. Spectrum logging (CSV format)
4. List mode logging (energy + timestamp)
5. File rotation, hot-swap support

Success: Spectrum and list mode files on SD, readable on PC

Phase 6: WiFi + Bluetooth (Week 7-8)

Goal: Wireless connectivity for data upload

Tasks:

1. WiFiConnectorWorker
2. WiFiUploaderHandler (Safecast API)
3. BluetoothWorker (BLE advertising)
4. BluetoothHandler (GATT server)
5. Test upload to tt.safecast.org
6. Test BLE streaming to smartphone

Success: Data uploads to Safecast, BLE connection to mobile app

Phase 7: Feature Parity + New Features (Week 9-10)

Goal: Full parity + enhancements

Tasks:

1. Software coincidence detection
2. Real-time clock (RTC)
3. Display handler (OLED/LCD, optional)
4. Power management (sleep modes)
5. OTA firmware updates over WiFi
6. Full test suite (unit + integration)

Success: All original features + WiFi OTA, web config, remote monitoring

Folder Structure

```
pomelo_esp32c3/
├─ platformio.ini
├─ pomelo_partitions.csv
├─ include/
│   ├── pomelo_pins.h           # GPIO definitions
│   ├── pomelo_config.h        # System config
│   └─ nvm_params.h            # Parameter structures (from original)
├─ src/
│   ├── main.cpp                # Setup/loop
│   ├── controller/
│   │   ├── pomelo_controller.h
│   │   └─ pomelo_controller.cpp
│   ├── workers/
│   │   ├── gamma_detector_worker.h/cpp
│   │   ├── hv_controller_worker.h/cpp
│   │   ├── temp_sensor_worker.h/cpp
│   │   ├── usb_serial_worker.h/cpp
│   │   └─ pomelo_storage.h/cpp
│   ├── handlers/
│   │   ├── sd_logger_handler.h/cpp
│   │   ├── wifi_uploader_handler.h/cpp
│   │   └─ bt_reporter_handler.h/cpp
│   ├── utils/
│   │   ├── command_interpreter.h/cpp
│   │   ├── ring_buffer.h
│   │   ├── spectral_deconv.cpp
│   │   ├── temp_sensors.cpp
│   │   └─ hv_control.cpp
│   └─ identifiers.h           # Worker/Handler IDs
└─ lib/
    └─ pMove/                  # Spectral deconvolution matrix
```

Critical Files

Reference Files (Read-Only)

- **bGeigieZen Controller:** bgeigiezen_firmware/controller.{h,cpp} - DeviceState pattern, worker registration
- **SensorReporter Library:** Worker.hpp, Handler.hpp - Base template classes
- **Original Pomelo:** PomeloCore/src/main.c - ISR timing, HV control, JSON protocol
- **Parameter Structures:** PomeloCore/src/nvm_params.h - Backwards compatibility

Files to Create

1. **src/controller/pomelo_controller.{h,cpp}** - Central controller with DeviceState
2. **src/workers/gamma_detector_worker.{h,cpp}** - ISR wrapper, spectral processing
3. **src/workers/hv_controller_worker.{h,cpp}** - HV regulation with temp compensation
4. **src/utils/command_interpreter.{h,cpp}** - JSON command compatibility layer

Risk Mitigation

Critical Risks

Risk	Mitigation
ISR timing too slow (>10µs)	Pre-allocate IRAM, optimize ISR, use Level 1 interrupt priority
ADC noise/accuracy	Calibrate with <code>esp_adc_cal</code> , hardware filtering
HV instability	Tune PID controller, increase PWM frequency
WiFi interference with ADC	Disable WiFi during critical measurements, shielded enclosure

Contingency Plans

If ISR timing fails:

- Use DMA for ADC sequencing (hardware control)
- Increase ESP32-C3 clock to 160MHz
- Simplify ISR, accept small timing drift

Testing Strategy

Unit Tests

- Per-worker testing with Unity framework
- Spectral deconvolution validation
- Ring buffer stress testing
- Parameter persistence tests

Integration Tests

- ISR timing verification (oscilloscope)
- Spectral accuracy (Cs-137 source comparison)
- Command protocol compatibility (Pomelo GUI)
- HV regulation (thermal chamber 0-40°C)
- WiFi upload reliability (1000 measurements)
- SD logging endurance (1GB fill test)

Performance Targets

Metric	Target	Verification
ISR duration	<10µs	Oscilloscope

Metric	Target	Verification
Max count rate	100 kHz	Function generator
HV stability	±0.1V	Multimeter
Temp comp accuracy	±0.5V over 40°C	Thermal chamber
Spectrum resolution	1024 bins	Cs-137 source
Command latency	<100ms	Serial monitor
WiFi upload rate	>1 Hz	Network analyzer

Sources

bGeigieZen:

- [GitHub Repository](#)
- [Documentation Site](#)
- [Safecast Device Page](#)

Architecture References:

- [M5Stack Hardware](#)
- [SensorReporter Library](#)

Summary

This plan provides a comprehensive roadmap for porting Pomelo firmware to ESP32-C3 using the proven bGeigieZen Worker/Handler/Controller architecture. Key innovations:

1. **ISR-Worker Split:** Critical timing in ISR, heavy processing deferred to worker
2. **Lock-Free Ring Buffer:** Safe ISR→worker communication without blocking
3. **100% Backwards Compatibility:** Existing JSON commands and NVM parameters preserved
4. **Phased Migration:** 7 phases from LED blink to full feature parity + enhancements
5. **New Capabilities:** WiFi upload, Bluetooth streaming, SD logging (not possible on ATSAML21)

The architecture is proven (bGeigieZen: 15,000+ field hours), and ESP32-C3 has sufficient performance (160MHz RISC-V, Level 1 interrupts) for critical ISR timing.

Estimated Timeline: 10 weeks with 1 developer